🤖 SJSU CyberAI Lab — Getting Started

NVIDIA Jetson Orin Nano · `sjsujetsontool`

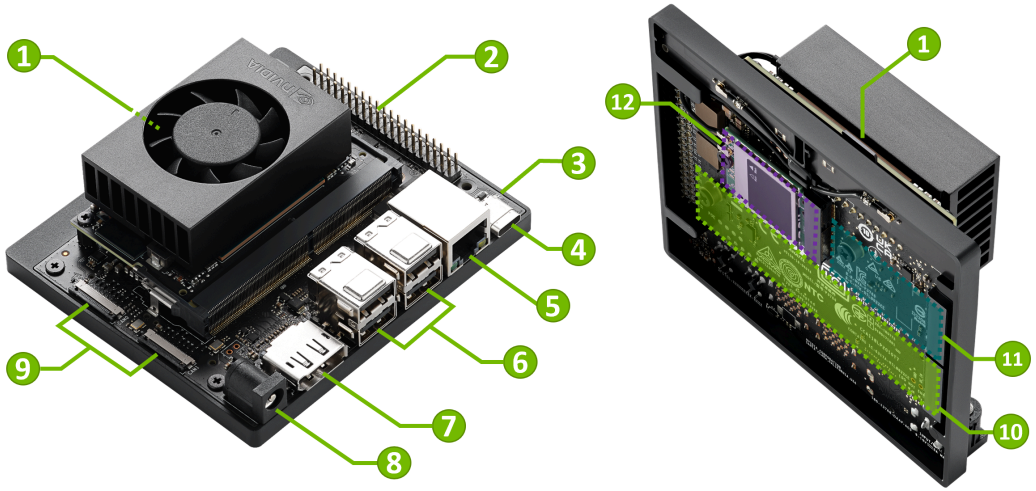
`San José State University · Edge AI`

A quick tour from first login to chatting with AI models. Follow along step by step.

🏕️ SJSU CyberAI Camp · 💻 github.com/lkk688/edgeai

1 At your lab station

- Turn on the monitor and **switch its input to DisplayPort** (the Jetson is connected via DisplayPort).
- You should see the **Jetson / Ubuntu login screen** with the green **NVIDIA** logo.



No login screen? Check the monitor source button and that the Jetson is powered on.

2 Log in as `student`

- On the login screen, pick the **`student`** account.
- Initial password: **`Sjsujetson2026`**

Change your password right away (open a Terminal):

```
passwd
# Current password: Sjsujetson2026
# New password: ********
```

> Keep your new password somewhere safe — you'll use it every session.

3 Your account & the container

The `student` account has **no** **`sudo`** (on purpose). Most AI tools run inside a prebuilt **container** that already has PyTorch, CUDA, llama.cpp and more.

```
sjsujetsontool shell      # quick way into the container (you're now "inside")
exit                      # leave the container
```

`sjsujetsontool` is **already installed**. Update it once at the start of the lab:

```
sjsujetsontool update     # refresh the tool + AI container
```

4 The shared `/Developer` folder

Inside the container, **`/Developer` is shared with the host** — files you create in one show up in the other. Put your work and models here.

- In the desktop, open the **Files** app and browse to **`/Developer`** to see the same files from the GUI (drag in datasets, open results, etc.).

```
ls /Developer            # same folder inside the container and on the host
```

5 Notebooks & scripts

JupyterLab — interactive notebooks in your browser:

```
sjsujetsontool jupyter          # serves JupyterLab on port 8888
```

Run a Python file inside the container (no need to enter the shell):

```
sjsujetsontool run /Developer/edgeAI/jetson/test.py
```

> Great for quick experiments and for running lab starter code in `/Developer`.

6 Run a local LLM 🧠

```
sjsujetsontool llama
```

Press **Enter** to accept the default **Qwen3.5-2B**, then choose **foreground** so you can **watch the llama.cpp log** (model load, tokens/sec) live in this terminal.

> It serves an OpenAI-style API at `http://localhost:8080`. Leave it running here.
>
> 📖 Details: Serving LLM via llama-server

7 Test it — the easy way

Open **another Terminal**. Instead of typing a long `curl`, let the tool build it for you:

```
sjsujetsontool curl
```

It asks a few questions — **LLM API** → host/port (Enter for `localhost:8080`), key (optional), your message, `max_tokens`, **stream?**, **thinking?** — then **prints the full curl and sends it**.

> 👍 Great for learning: it shows you the exact `curl` command it ran.

8 Test it — raw `curl` (two modes)

Non-streaming — wait for the full answer (thinking off = short & fast):

```
curl http://localhost:8080/v1/chat/completions -H "Content-Type: application/json" -d '{
  "messages":[{"role":"user","content":"Explain Nvidia Jetson in 2 sentences."}],
  "max_tokens":150, "chat_template_kwargs":{"enable_thinking":false} }'
```

Streaming — tokens appear live (add `-N` and `"stream":true`):

```
curl -N http://localhost:8080/v1/chat/completions -H "Content-Type: application/json" -d '{
  "messages":[{"role":"user","content":"Explain Nvidia Jetson in 2 sentences."}],
  "max_tokens":150, "stream":true, "chat_template_kwargs":{"enable_thinking":false} }'
```

📖 Full curl guide (SSE format, options): HTTP API docs

9 👀 Vision — ask about an image

Qwen3.5-2B and Gemma-4 are **multimodal**. With the local server running (step 6), attach an image using `sjsujetsontool curl`:

```
sjsujetsontool curl       # LLM API → at "Attach an image?" enter a path
# image: /Developer/models/bus.jpg   message: "What's in this image? How many people?"
```

Or run the ready-made sample (sends the OpenAI `image_url` format):

```
sjsujetsontool run /Developer/edgeAI/jetson/jetson-llm/vision_test.py \
  --image /Developer/models/bus.jpg -p "Describe this image."
# → VISION REPLY: A blue city bus with several people standing beside it.
```

📖 Code: `jetson/jetson-llm/vision_test.py`

10 Cloud models (optional): NVIDIA API

```
sjsujetsontool setup-nvapi      # paste your key (from build.nvidia.com)
sjsujetsontool nv-chat          # chat with cloud Nemotron models + speed stats
```

> Get a free key at **build.nvidia.com** → sign in → open a model → *Get API Key*.
> 🔒 Your key is saved in your **private** `~/.env.local`. **Don't share that file** — you can delete it anytime to remove your keys.

11 `sjsujetsontool chat` — one client, many backends

One terminal client for **local and cloud** models; switch any time with `/server`.

Example — the local model (no key needed):

```
sjsujetsontool llama       # 1) start the local server (default Qwen3.5-2B)
sjsujetsontool chat        # 2) (another terminal) → pick "1) Local Jetson llama.cpp"
```

Inside the chat, handy commands: `/think on|off` · `/temp 0.7` · `/save` · `/server` · `/exit`.

12 chat — adding cloud backends

The first time you pick a cloud backend, it **prompts for the key and saves it** to `~/.env.local`:

Backend	Get a key at	Saved as
NVIDIA	build.nvidia.com → *Get API Key* (free)	`NVIDIA_API_KEY`
OpenAI	platform.openai.com/api-keys (needs billing)	`OPENAI_API_KEY`
Anthropic	console.anthropic.com/settings/keys (needs credit)	`ANTHROPIC_API_KEY`

```
sjsujetsontool chat        # pick 2 NVIDIA / 3 OpenAI / 4 Anthropic → paste key once → chat
```

🔒 `~/.env.local` is **your private file** — don't share it; delete it anytime to remove your keys.

📖 Details: one chat client, many backends

13 🖥️ A web chat UI (Gradio)

Prefer a browser? The sample UI uses the **same backends**, plus **file & image** upload:

```
sjsujetsontool gradio        # installs deps first run → open http://localhost:7860
```

- Pick **Local / NVIDIA / OpenAI / Anthropic** in the UI (keys read from `~/.env.local`).
- Drop in an **image** to test vision, or a **text file** to ask about its contents.

📖 Code: edgeLLM/gradio_chat_ui.py

> 🚀 Want a full **web app**? Continue with the **Next.js + Nemotron slides** ▶️
> · full lab

🎉 You're set!

DisplayPort → log in → `update` → `shell` → `llama` → `chat`

Full handbook (CUDA, YOLO, ROS 2, robotics, and more):
lkk688.github.io/edgeAI

Helpful any time: `sjsujetsontool list` · `sjsujetsontool dockerfix`